Persistent block device naming

(/dev/disk/ : by-id, by-label, by-partlabel, by-partuuid, by-path, by-uuid)

Persistent block device naming

This article describes how to use persistent names for your **block devices**. This has been made possible by the introduction of **udev** and has some advantages over bus-based naming. If your machine has more than one SATA, SCSI or IDE disk controller, the order in which their corresponding device nodes are added is arbitrary. This may result in device names like `/dev/sda` and `/dev/sdb` switching around on each boot, culminating in an unbootable system, kernel panic, or a block device disappearing. Persistent naming solves these issues.

Note:

- Persistent naming has limits that are out-of-scope in this article. For example, while **mkinitcpio** may support a method, systemd may impose its own limits (e.g. **FS#42884**) on naming it can process during boot.
- This article is not relevant for **LVM** logical volumes as the `/dev/VolumeGroupName/LogicalVolumeName` device paths are persistent.

note: Remember:
#to list all block devices
$ lsblk

note:

- by-partuuid is only available to gpt disks
- by-uuid is available to both mbr and gpt disks (so its preferred)

Contents [hide]


```
└─ $ls /dev/disk
by-id  by-label  by-partlabel  by-partuuid  by-path  by-uuid
```

#[SUMMARY] : In order to get a devices persistent block device naming information [FOR UUID SPECIFICALLY, PREFERRED]:

$ lsblk -f
$ lsblk -f /dev/<sdXY device>
$ lsblk -f | grep sdX
$ ls -l /dev/disk/by-uuid/
$ ls -l /dev/disk/by-uuid/ | grep <sdXY device>

$ ls -l /dev/disk/by-<bydirectory>
$ ls -l /dev/disk/by-<bydirectory> | grep sdX

Persistent naming methods

There are four different schemes for persistent naming: **by-label**, **by-uuid**, **by-id and by-path**. For those using disks with **GUID Partition Table (GPT)**, two additional schemes can be used **by-partlabel** and **by-partuuid**. You can also use **static device names by using Udev**.

The directories in `/dev/disk/` are created and destroyed dynamically, depending on whether there are devices in them.

Note: Beware that **Disk cloning** creates two different disks with the same name.

The following sections describe what the different persistent naming methods are and how they are used.

The **lsblk** command can be used for viewing graphically the first persistent schemes:

```
$ lsblk -f
```

```
NAME        FSTYPE LABEL      UUID                                 MOUNTPOINT
sda
├─sda1      vfat              CBB6-24F2                            /boot
├─sda2      ext4   Arch Linux 0a3407de-014b-458b-b5c1-848e92a327a3 /
├─sda3      ext4   Data       b411dc99-f0a0-4c87-9e05-184977be8539 /home
└─sda4      swap              f9fe0b69-a280-415d-a03a-a32752370dee [SWAP]
mmcblk0
└─mmcblk0p1 vfat              F4CA-5D75
```

For those using **GPT**, use the `blkid` command instead. The latter is more convenient for scripts, but more difficult to read.

```
# blkid
```

```
/dev/sda1: UUID="CBB6-24F2" TYPE="vfat" PARTLABEL="EFI system partition" PARTUUID="d0d0d110-0a71-4ed6-936a-304969ea36af"
/dev/sda2: LABEL="Arch Linux" UUID="0a3407de-014b-458b-b5c1-848e92a327a3" TYPE="ext4" PARTLABEL="GNU/Linux" PARTUUID="98a81274-10f7-40db-872a-03df04
8df366"
/dev/sda3: LABEL="Data" UUID="b411dc99-f0a0-4c87-9e05-184977be8539" TYPE="ext4" PARTLABEL="Home" PARTUUID="7280201c-fc5d-40f2-a9b2-466611d3d49e"
/dev/sda4: UUID="f9fe0b69-a280-415d-a03a-a32752370dee" TYPE="swap" PARTLABEL="Swap" PARTUUID="039b6c1c-7553-4455-9537-1befbc9fbc5b"
/dev/mmcblk0: PTUUID="0003e1e5" PTTYPE="dos"
/dev/mmcblk0p1: UUID="F4CA-5D75" TYPE="vfat" PARTUUID="0003e1e5-01"
```

by-label

Almost every **file system type** can have a label. All your volumes that have one are listed in the `/dev/disk/by-label` directory.

```
$ ls -l /dev/disk/by-label
```

```
total 0
lrwxrwxrwx 1 root root 10 May 27 23:31 Data -> ../../sda3
lrwxrwxrwx 1 root root 10 May 27 23:31 Arch\x20Linux -> ../../sda2
```

Most file systems support setting the label upon file system creation, see the **man page** of the relevant `mkfs.*` utility. For some file systems it is also possible to change the labels. Following are some methods for changing labels on common file systems:

swap
`swaplabel -L "new label" /dev/XXX` using **util-linux**

ext2/3/4
`e2label /dev/XXX "new label"` using **e2fsprogs**

btrfs
`btrfs filesystem label /dev/XXX "new label"` using **btrfs-progs**

reiserfs
`reiserfstune -l "new label" /dev/XXX` using **reiserfsprogs**

jfs
`jfs_tune -L "new label" /dev/XXX` using **jfsutils**

xfs
`xfs_admin -L "new label" /dev/XXX` using **xfsprogs**

fat/vfat
`fatlabel /dev/XXX "new label"` using **dosfstools**
`mlabel -i /dev/XXX ::"new label"` using **mtools**

exfat

`tune.exfat -L "new label" /dev/XXX` using **exfatprogs**

`exfatlabel /dev/XXX "new label"` using **exfatprogs** or **exfat-utils**

ntfs

`ntfslabel /dev/XXX "new label"` using **ntfs-3g**

udf

`udflabel /dev/XXX "new label"` using **udftools**

crypto_LUKS (LUKS2 only)

`cryptsetup config --label="new label" /dev/XXX` using **cryptsetup**

The label of a device can be obtained with *lsblk*:

```
$ lsblk -dno LABEL /dev/sda2
```

```
Arch Linux
```

Or with *blkid*:

```
# blkid -s LABEL -o value /dev/sda2
```

```
Arch Linux
```

Note:

- The file system must not be mounted to change its label. For the root file system this can be accomplished by booting from another volume.
- Labels have to be unambiguous to prevent any possible conflicts.
- Labels can be up to 16 characters long.
- Since the label is a property of the filesystem, it is not suitable for addressing a single RAID device persistently.
- When using encrypted containers with **dm-crypt**, the labels of filesystems inside of containers are not available while the container is locked/encrypted.

by-uuid

UUID is a mechanism to give each **filesystem** a unique identifier. These identifiers are generated by filesystem utilities (e.g. `mkfs.*`) when the device gets formatted and are designed so that collisions are unlikely. All GNU/Linux filesystems (including swap and LUKS headers of raw encrypted devices) support UUID. FAT, exFAT and NTFS filesystems do not support UUID, but are still listed in `/dev/disk/by-uuid/` with a shorter UID (unique identifier):

```
$ ls -l /dev/disk/by-uuid/
```

```
total 0
lrwxrwxrwx 1 root root 10 May 27 23:31 0a3407de-014b-458b-b5c1-848e92a327a3 -> ../../sda2
lrwxrwxrwx 1 root root 10 May 27 23:31 b411dc99-f0a0-4c87-9e05-184977be8539 -> ../../sda3
lrwxrwxrwx 1 root root 10 May 27 23:31 CBB6-24F2 -> ../../sda1
lrwxrwxrwx 1 root root 10 May 27 23:31 f9fe0b69-a280-415d-a03a-a32752370dee -> ../../sda4
lrwxrwxrwx 1 root root 10 May 27 23:31 F4CA-5D75 -> ../../mmcblk0p1
```

The UUID of a device can be obtained with *lsblk*:

```
$ lsblk -dno UUID /dev/sda1
```

```
CBB6-24F2
```

Or with *blkid*:

```
# blkid -s UUID -o value /dev/sda1
```

```
CBB6-24F2
```

The advantage of using the UUID method is that it is much less likely that name collisions occur than with labels. Further, it is generated automatically on creation of the filesystem. It will, for example, stay unique even if the device is plugged into another system (which may perhaps have a device with the same label).

The disadvantage is that UUIDs make long code lines hard to read and break formatting in many configuration files (e.g. **fstab** or **crypttab**). Also every time a volume is reformatted a new UUID is generated and configuration files have to get manually adjusted.

Tip: In case your swap does not have an UUID assigned, you will need to reset it using the **mkswap** utility.

by-id and by-path

`by-id` creates a unique name depending on the hardware serial number, `by-path` depending on the shortest physical path (according to sysfs). Both contain strings to indicate which subsystem they belong to (i.e. `pci-` for `by-path`, and `ata-` for `by-id`), so they are linked to the hardware controlling the device. This implies different levels of persistence: the `by-path` will already change when the device is plugged into a different port of the controller, the `by-id` will change when the device is plugged into a port of a hardware controller subject to another subsystem. [1] Thus, both are not suitable to achieve persistent naming tolerant to hardware changes.

However, both provide important information to find a particular device in a large hardware infrastructure. For example, if you do not manually assign persistent labels (`by-label` or `by-partlabel`) and keep a directory with hardware port usage, `by-id` and `by-path` can be used to find a particular device.[2] [3]

`by-id` also creates **World Wide Name** links of storage devices that support it. Unlike other `by-id` links, WWNs are fully persistent and will not change depending on the used subsystem.

This article or section needs expansion.

Reason: Explain and provide examples with `/dev/disk/by-id/nvme-eui.*` links[4]. Will the WWID of NVME devices always start with `eui.`? (Discuss in **Talk:Persistent block device naming#**)

Note: `by-id` and `by-path` links can only be considered persistent for disks, not partitions. Partitions will be referenced by their number in the partition table and that can change if the partitions are reordered.

```
$ ls -l /dev/disk/by-id/
```

```
total 0
lrwxrwxrwx 1 root root 10 May 27 23:31 ata-WDC_WD2500BEVT-22ZCT0_WD-WXE908VF0470 -> ../../sda
lrwxrwxrwx 1 root root 10 May 27 23:31 ata-WDC_WD2500BEVT-22ZCT0_WD-WXE908VF0470-part1 -> ../../sda1
lrwxrwxrwx 1 root root 10 May 27 23:31 ata-WDC_WD2500BEVT-22ZCT0_WD-WXE908VF0470-part2 -> ../../sda2
lrwxrwxrwx 1 root root 10 May 27 23:31 ata-WDC_WD2500BEVT-22ZCT0_WD-WXE908VF0470-part3 -> ../../sda3
lrwxrwxrwx 1 root root 10 May 27 23:31 ata-WDC_WD2500BEVT-22ZCT0_WD-WXE908VF0470-part4 -> ../../sda4
lrwxrwxrwx 1 root root 10 May 27 23:31 mmc-SD32G_0x0040006d -> ../../mmcblk0
lrwxrwxrwx 1 root root 10 May 27 23:31 mmc-SD32G_0x0040006d-part1 -> ../../mmcblk0p1
lrwxrwxrwx 1 root root 10 May 27 23:31 wwn-0x60015ee0000b237f -> ../../sda
lrwxrwxrwx 1 root root 10 May 27 23:31 wwn-0x60015ee0000b237f-part1 -> ../../sda1
lrwxrwxrwx 1 root root 10 May 27 23:31 wwn-0x60015ee0000b237f-part2 -> ../../sda2
```

```
lrwxrwxrwx 1 root root 10 May 27 23:31 wwn-0x60015ee0000b237f-part3 -> ../../sda3
lrwxrwxrwx 1 root root 10 May 27 23:31 wwn-0x60015ee0000b237f-part4 -> ../../sda4
```

```
$ ls -l /dev/disk/by-path/
```

```
total 0
lrwxrwxrwx 1 root root 10 May 27 23:31 pci-0000:00:1f.2-ata-1 -> ../../sda
lrwxrwxrwx 1 root root 10 May 27 23:31 pci-0000:00:1f.2-ata-1-part1 -> ../../sda1
lrwxrwxrwx 1 root root 10 May 27 23:31 pci-0000:00:1f.2-ata-1-part2 -> ../../sda2
lrwxrwxrwx 1 root root 10 May 27 23:31 pci-0000:00:1f.2-ata-1-part3 -> ../../sda3
lrwxrwxrwx 1 root root 10 May 27 23:31 pci-0000:00:1f.2-ata-1-part4 -> ../../sda4
lrwxrwxrwx 1 root root 10 May 27 23:31 pci-0000:07:00.0-platform-rtsx_pci_sdmmc.0 -> ../../mmcblk0
lrwxrwxrwx 1 root root 10 May 27 23:31 pci-0000:07:00.0-platform-rtsx_pci_sdmmc.0-part1 -> ../../mmcblk0p1
```

by-partlabel

Note: This method only concerns disks with **GUID Partition Table (GPT)**.

GPT partition labels can be defined in the header of the **partition entry** on GPT disks.

This method is very similar to the **filesystem labels**, except the partition labels do not get affected if the file system on the partition is changed.

All partitions that have partition labels are listed in the `/dev/disk/by-partlabel` directory.

```
ls -l /dev/disk/by-partlabel/
```

```
total 0
lrwxrwxrwx 1 root root 10 May 27 23:31 EFI\x20system\x20partition -> ../../sda1
lrwxrwxrwx 1 root root 10 May 27 23:31 GNU\x2fLinux -> ../../sda2
lrwxrwxrwx 1 root root 10 May 27 23:31 Home -> ../../sda3
lrwxrwxrwx 1 root root 10 May 27 23:31 Swap -> ../../sda4
```

The partition label of a device can be obtained with *lsblk*:

```
$ lsblk -dno PARTLABEL /dev/sda1
```

```
EFI system partition
```

Or with *blkid*:

```
# blkid -s PARTLABEL -o value /dev/sda1
```

```
EFI system partition
```

Note:

- GPT partition labels also have to be different to avoid conflicts. To change your partition label, you can use **gdisk** or the ncurses-based version **cgdisk**. Both are available from the `gptfdisk` package. See **Partitioning#Partitioning tools**.
- According to the specification, GPT partition labels can be up to 72 characters long.

by-partuuid

Like **GPT partition labels**, GPT partition UUIDs are defined in the **partition entry** on GPT disks.

MBR does not support partition UUIDs, but Linux[5] and software using libblkid[6] (e.g. udev[7]) are capable of generating pseudo PARTUUIDs for MBR partitions. The format is `SSSSSSSS-PP`, where `SSSSSSSS` is a zero-filled 32-bit **MBR disk signature**, and `PP` is a zero-filled partition number in hexadecimal form. Unlike a regular PARTUUID of a GPT partition, MBR's pseudo PARTUUID can change if the partition number changes.

The dynamic directory is similar to other methods and, like **filesystem UUIDs**, using UUIDs is preferred over labels.

```
ls -l /dev/disk/by-partuuid/
```

```
total 0
lrwxrwxrwx 1 root root 10 May 27 23:31 0003e1e5-01 -> ../../mmcblk0p1
lrwxrwxrwx 1 root root 10 May 27 23:31 039b6c1c-7553-4455-9537-1befbc9fbc5b -> ../../sda4
lrwxrwxrwx 1 root root 10 May 27 23:31 7280201c-fc5d-40f2-a9b2-466611d3d49e -> ../../sda3
lrwxrwxrwx 1 root root 10 May 27 23:31 98a81274-10f7-40db-872a-03df048df366 -> ../../sda2
lrwxrwxrwx 1 root root 10 May 27 23:31 d0d0d110-0a71-4ed6-936a-304969ea36af -> ../../sda1
```

The partition UUID of a device can be obtained with *lsblk*:

```
$ lsblk -dno PARTUUID /dev/sda1
```

```
d0d0d110-0a71-4ed6-936a-304969ea36af
```

Or with *blkid*:

```
# blkid -s PARTUUID -o value /dev/sda1
```

```
d0d0d110-0a71-4ed6-936a-304969ea36af
```

Static device names with udev

See **udev#Setting static device names**.

Using persistent naming

There are various applications that can be configured using persistent naming. Following are some examples of how to configure them.

fstab

See the main article: **fstab#Identifying filesystems**.

Kernel parameters

To use persistent names in **kernel parameters**, the following prerequisites must be met. On a standard installation following the installation guide both prerequisites are met:

- You are using an **initramfs** image that has **udev** in it.
  - For **mkinitcpio**, enable either the udev or systemd hook in `/etc/mkinitcpio.conf`

The location of the root filesystem is given by the parameter `root` on the kernel command line. The kernel command line is configured from the **boot loader**, see **Kernel parameters#Configuration**. To change to persistent device naming, only change the parameters which specify block devices, e.g. `root` and `resume`, while leaving other parameters as is. Various naming schemes are supported:

Persistent device naming **using label** and the `LABEL=` format, in this example `Arch Linux` is the LABEL of the root file system.

```
root="LABEL=Arch Linux"
```

Persistent device naming **using UUID** and the `UUID=` format, in this example `0a3407de-014b-458b-b5c1-848e92a327a3` is the UUID of the root file system.

```
root=UUID=0a3407de-014b-458b-b5c1-848e92a327a3
```

Persistent device naming **using disk id** and the `/dev` path format, in this example `wwn-0x60015ee0000b237f-part2` is the id of the root partition.

```
root=/dev/disk/by-id/wwn-0x60015ee0000b237f-part2
```

Persistent device naming **using GPT partition UUID** and the `PARTUUID=` format, in this example `98a81274-10f7-40db-872a-03df048df366` is the PARTUUID of the root partition.

```
root=PARTUUID=98a81274-10f7-40db-872a-03df048df366
```

Persistent device naming **using GPT partition label** and the `PARTLABEL=` format, in this example `GNU/Linux` is the PARTLABEL of the root partition.

```
root="PARTLABEL=GNU/Linux"
```

#[SUMMARY] : In order to get a devices persistent block device naming information [FOR UUID SPECIFICALLY, PREFERRED]:

$ lsblk -f
$ lsblk -f /dev/<sdXY device>
$ ls -l /dev/disk/by-uuid/
$ ls -l /dev/disk/by-uuid/ | grep <sdXY device>

$ lsblk -f | grep sdX

$ ls -l /dev/disk/by-<bydirectory>
$ ls -l /dev/disk/by-<bydirectory> | grep sdX

ex; $ ls -l /dev/disk/by-partuuid
ex2; $ ls -l /dev/disk/by-partuuid |grep sda

RED HAT PERSISTENT NAMING
(/dev/disk/ : by-id, by-label, by-partlabel, by-partuuid, by-path, by-uuid)

PERSISTENT NAMING

Red Hat Enterprise Linux provides a number of ways to identify storage devices. It is important to use the correct option to identify each device when used in order to avoid inadvertently accessing the wrong device, particularly when installing to or reformatting drives.

Major and Minor Numbers of Storage Devices (sd<letter(s)>[number(s)])

Storage devices managed by the `sd` driver are identified internally by a collection of major device numbers and their associated minor numbers. The major device numbers used for this purpose are not in a contiguous range. Each storage device is represented by a major number and a range of minor numbers, which are used to identify either the entire device or a partition within the device. There is a direct association between the major and minor numbers allocated to a device and numbers in the form of `sd<letter(s)>[number(s)]`. Whenever the `sd` driver detects a new device, an available major number and minor number range is allocated. Whenever a device is removed from the operating system, the major number and minor number range is freed for later reuse.

The major and minor number range and associated `sd` names are allocated for each device when it is detected. This means that the association between the major and minor number range and associated `sd` names can change if the order of device detection changes. Although this is unusual with some hardware configurations (for example, with an internal SCSI controller and disks that have their SCSI target ID assigned by their physical location within a chassis), it can nevertheless occur. Examples of situations where this can happen are as follows:

- A disk may fail to power up or respond to the SCSI controller. This will result in it not being detected by the normal device probe. The disk will not be accessible to the system and subsequent devices will have their major and minor number range, including the associated `sd` names shifted down. For example, if a disk normally referred to as `sdb` is not detected, a disk that is normally referred to as `sdc` would instead appear as `sdb`.
- A SCSI controller (host bus adapter, or HBA) may fail to initialize, causing all disks connected to that HBA to not be detected. Any disks connected to subsequently probed HBAs would be assigned different major and minor number ranges, and different associated `sd` names.
- The order of driver initialization could change if different types of HBAs are present in the system. This would cause the disks connected to those HBAs to be detected in a different order. This can also occur if HBAs are moved to different PCI slots on the system.
- Disks connected to the system with Fibre Channel, iSCSI, or FCoE adapters might be inaccessible at the time the storage devices are probed, due to a storage array or intervening switch being powered off, for example. This could occur when a system reboots after a power failure, if the storage array takes longer to come online than the system take to boot. Although some Fibre Channel drivers support a mechanism to specify a persistent SCSI target ID to WWPN mapping, this will not cause the major and minor number ranges, and the associated `sd` names to be reserved, it will only provide consistent SCSI target ID numbers.

These reasons make it undesirable to use the major and minor number range or the associated `sd` names when referring to devices, such as in the `/etc/fstab` file. There is the possibility that the wrong device will be mounted and data corruption could result.

Occasionally, however, it is still necessary to refer to the `sd` names even when another mechanism is used (such as when errors are reported by a device). This is because the Linux kernel uses `sd` names (and also SCSI host/channel/target/LUN tuples) in kernel messages regarding the device.

25.8.2. World Wide Identifier (WWID)

The *World Wide Identifier* (WWID) can be used in reliably identifying devices. It is a persistent, system-independent ID that the SCSI Standard requires from all SCSI devices. The WWID identifier is guaranteed to be unique for every storage device, and independent of the path that is used to access the device.

This identifier can be obtained by issuing a SCSI Inquiry to retrieve the *Device Identification Vital Product Data* (page `0x83`) or *Unit Serial Number* (page `0x80`). The mappings from these WWIDs to the current `/dev/sd` names can be seen in the symlinks maintained in the `/dev/disk/by-id/` directory.

Example 25.4. WWID

For example, a device with a page `0x83` identifier would have:

```
scsi-3600508b400105e210000900000490000 -> ../../sda
```

Or, a device with a page `0x80` identifier would have:

```
scsi-SSEAGATE_ST373453LW_3HW1RHM6 -> ../../sda
```

Red Hat Enterprise Linux automatically maintains the proper mapping from the WWID-based device name to a current `/dev/sd` name on that system. Applications can use the `/dev/disk/by-id/` name to reference the data on the disk, even if the path to the device changes, and even when accessing the device from different systems.

If there are multiple paths from a system to a device, **DM Multipath** uses the WWID to detect this. **DM Multipath** then presents a single "pseudo-device" in the `/dev/mapper/wwid` directory, such as `/dev/mapper/3600508b400105df70000e00000ac0000`.

The command `multipath -l` shows the mapping to the non-persistent identifiers: *`Host:Channel:Target:LUN`*, `/dev/sd` name, and the `major:minor` number.

```
3600508b400105df70000e00000ac0000 dm-2 vendor,product
[size=20G][features=1 queue_if_no_path][hwhandler=0][rw]
\_ round-robin 0 [prio=0][active]
 \_ 5:0:1:1 sdc 8:32  [active][undef]
 \_ 6:0:1:1 sdg 8:96  [active][undef]
\_ round-robin 0 [prio=0][enabled]
 \_ 5:0:0:1 sdb 8:16  [active][undef]
 \_ 6:0:0:1 sdf 8:80  [active][undef]
```

DM Multipath automatically maintains the proper mapping of each WWID-based device name to its corresponding `/dev/sd` name on the system. These names are persistent across path changes, and they are consistent when accessing the device from different systems.

When the `user_friendly_names` feature (of **DM Multipath**) is used, the WWID is mapped to a name of the form `/dev/mapper/mpathn`. By default, this mapping is maintained in the file `/etc/multipath/bindings`. These `mpathn` names are persistent as long as that file is maintained.

Important

If you use `user_friendly_names`, then additional steps are required to obtain consistent names in a cluster. Refer to the Consistent Multipath Device Names in a Cluster section in the DM Multipath book.

In addition to these persistent names provided by the system, you can also use `udev` rules to implement persistent names of your own, mapped to the WWID of the storage.

25.8.3. Device Names Managed by the `udev` Mechanism in `/dev/disk/by-*`

The `udev` mechanism consists of three major components:

The kernel
: Generates events that are sent to user space when devices are added, removed, or changed.

The `udevd` service
: Receives the events.

The `udev` rules
: Specifies the action to take when the `udev` service receives the kernel events.

This mechanism is used for all types of devices in Linux, not just for storage devices. In the case of storage devices, Red Hat Enterprise Linux contains `udev` rules that create symbolic links in the `/dev/disk/` directory allowing storage devices to be referred to by their contents, a unique identifier, their serial number, or the hardware path used to access the device.

`/dev/disk/by-label/`

Entries in this directory provide a symbolic name that refers to the storage device by a label in the contents (that is, the data) stored on the device. The **blkid** utility is used to read data from the device and determine a name (that is, a label) for the device. For example:

```
/dev/disk/by-label/Boot
```

Note

The information is obtained from the contents (that is, the data) on the device so if the contents are copied to another device, the label will remain the same.

The label can also be used to refer to the device in `/etc/fstab` using the following syntax:

```
LABEL=Boot
```

`/dev/disk/by-uuid/`

Entries in this directory provide a symbolic name that refers to the storage device by a unique identifier in the contents (that is, the data) stored on the device. The **blkid** utility is used to read data from the device and obtain a unique identifier (that is, the UUID) for the device. For example:

```
UUID=3e6be9de-8139-11d1-9106-a43f08d823a6
```

`/dev/disk/by-id/`

Entries in this directory provide a symbolic name that refers to the storage device by a unique identifier (different from all other storage devices). The identifier is a property of the device but is not stored in the contents (that is, the data) on the devices. For example:

```
/dev/disk/by-id/scsi-3600508e000000000ce506dc50ab0ad05
```

```
/dev/disk/by-id/wwn-0x600508e000000000ce506dc50ab0ad05
```

The id is obtained from the world-wide ID of the device, or the device serial number. The `/dev/disk/by-id/` entries may also include a partition number. For example:

```
/dev/disk/by-id/scsi-3600508e000000000ce506dc50ab0ad05-part1
```

```
/dev/disk/by-id/wwn-0x600508e000000000ce506dc50ab0ad05-part1
```

`/dev/disk/by-path/`

Entries in this directory provide a symbolic name that refers to the storage device by the hardware path used to access the device, beginning with a reference to the storage controller in the PCI hierarchy, and including the SCSI host, channel, target, and LUN numbers and, optionally, the partition number. Although these names are preferable to using major and minor numbers or `sd` names, caution must be used to ensure that the target numbers do not change in a Fibre Channel SAN environment (for example, through the use of persistent binding) and that the use of the names is updated if a host adapter is moved to a different PCI slot. In addition, there is the possibility that the SCSI host numbers could change if a HBA fails to probe, if drivers are loaded in a different order, or if a new HBA is installed on the system. An example of by-path listing is:

```
/dev/disk/by-path/pci-0000:03:00.0-scsi-0:1:0:0
```

The `/dev/disk/by-path/` entries may also include a partition number, such as:

```
/dev/disk/by-path/pci-0000:03:00.0-scsi-0:1:0:0-part1
```

25.8.3.1. Limitations of the `udev` Device Naming Convention

The following are some limitations of the `udev` naming convention.

- It is possible that the device may not be accessible at the time the query is performed because the `udev` mechanism may rely on the ability to query the storage device when the `udev` rules are processed for a `udev` event. This is more likely to occur with Fibre Channel, iSCSI or FCoE storage devices when the device is not located in the server chassis.

- The kernel may also send `udev` events at any time, causing the rules to be processed and possibly causing the `/dev/disk/by-*/` links to be removed if the device is not accessible.
- There can be a delay between when the `udev` event is generated and when it is processed, such as when a large number of devices are detected and the user-space `udevd` service takes some amount of time to process the rules for each one). This could cause a delay between when the kernel detects the device and when the `/dev/disk/by-*/` names are available.
- External programs such as **blkid** invoked by the rules may open the device for a brief period of time, making the device inaccessible for other uses.

25.8.3.2. Modifying Persistent Naming Attributes

Although `udev` naming attributes are persistent, in that they do not change on their own across system reboots, some are also configurable. You can set custom values for the following persistent naming attributes:

- `UUID`: file system UUID
- `LABEL`: file system label

Because the `UUID` and `LABEL` attributes are related to the file system, the tool you need to use depends on the file system on that partition.

- To change the `UUID` or `LABEL` attributes of an XFS file system, unmount the file system and then use the **xfs_admin** utility to change the attribute:

```
# umount /dev/device
# xfs_admin [-U new_uuid] [-L new_label] /dev/device
# udevadm settle
```

- To change the `UUID` or `LABEL` attributes of an ext4, ext3, or ext2 file system, use the **tune2fs** utility:

```
# tune2fs [-U new_uuid] [-L new_label] /dev/device
# udevadm settle
```

Replace *new_uuid* with the UUID you want to set; for example, `1cdfbc07-1c90-4984-b5ec-f61943f5ea50`. Replace *new_label* with a label; for example, `backup_data`.

Note

Changing `udev` attributes happens in the background and might take a long time. The `udevadm settle` command waits until the change is fully registered, which ensures that your next command will be able to utilize the new attribute correctly.

You should also use the command after creating new devices; for example, after using the **parted** tool to create a partition with a custom `PARTUUID` or `PARTLABEL` attribute, or after creating a new file system.